

Interconnects and Communication

Parallel hardware architectures are generally divided into Shared Memory (SM) and Distributed Memory (DM) systems.

- Shared Memory Interconnects: CPUs communicate by accessing shared data in a common memory system. Interconnects types include Buses (serial or parallel) and Crossbars, which use $O(m^2)$ switches and are non-blocking, allowing simultaneous memory access if different modules are targeted.

- Distributed Memory Interconnects: Systems can use Static (Direct) or Dynamic (Indirect) interconnects.

- * Direct topologies: Examples include 1D arrays, meshes, rings, toroidal meshes, and hypercubes. Hypercubes are inductively built and use E-cube routing (XORing source and destination ranks to determine the next hop along the dimension of the least-significant differing bit).

- * Indirect topologies: These utilize switches and include Fat Trees (common in Infiniband), Dragonfly (low diameter, used in systems like Frontier), and Omega Networks (multistage networks using a perfect shuffle pattern).

• Network Metrics: Performance is evaluated via diameter (distance between the two farthest nodes), bisection width (minimum links to cut to divide the network in half), arc connectivity, cost (number of links/switches), and bandwidth.

Communication Cost

Total message transfer time involves Startup time (t_s), Per-hop time (t_h), and Per-word transfer time (t_w).

* Store-and-Forward: The message is buffered completely at each hop being forwarded.

* Packet Routing: Messages are broken into packets and pipelined, introducing header overhead but improving performance through parallelism.

* Cut-Through Routing: Uses small packets called flits (flow control digits) that are not buffered at intermediate nodes, leading to a simplified cost model:

$$T_{\text{comm}} = t_s + t_w m$$

Mapping and Cache Coherence

* Mapping: Vital to performance, mapping involves ~~assign~~ assigning tasks to cores. This can be done via affinity (binding tasks to specific CPUs) or by embedding

one topology (like a linear array) into another (like a mesh) while trying to minimize congestion.

* Cache coherence: Ensures all cores see a consistent value for memory locations.

- Snooping: Cores monitor a shared bus to invalidate outdated local cache lines.

- Directory-based: A data structure tracks the status (Dirty, Shared, Invalid) of cache lines.

- False Sharing: A performance hazard where cores update different elements in the same cache line, causing unnecessary invalidations. This can be fixed using alignas to ensure elements reside in separate cache lines.

CUDA and GPU Architecture

NVIDIA's CUDA platform exposes GPU parallelism for general computing.

- GPU Architecture: GPUs contain Streaming Multiprocessors (SMs), each housing multiple Streaming Processors (SP) or CUDA cores (effectively ALUs). Memory is split between fast on-chip Local/Shared memory and slower Device memory (DRAM).

- Programming Model: Features a Host (CPU) and Device (GPU). Data management uses `cudaMalloc`, `cudaFree`, and `cudaMemcpy` (which is blocking) or `cudaMemcpyAsync`.

- Kernels and Execution: Kernels (marked - global -) are GPU functions launched from the Host using the configuration `<<< numBlocks, threadsPerBlock >>>`. Threads identify their work via `blockIdx.x` and `threadIdx.x`.

- Cooperation: Threads within a block share data via Shared Memory and synchronize using `__syncthreads()`.

Parallel Software and Shared Memory Programming

- Threads: Systems may use dynamic threads (created on demand) or static threads (thread pool). Programming issues include race conditions and non-determinism.

- Synchronization: Primitives include Mutexes (for critical sections), Condition Variables, and Semaphores. Atomics allow for lock-free updates and are used to implement efficient busy-waiting (spinning).

- Foster's Methodology: A design framework consisting of four steps: Partitioning (into tasks), Communication, Agglomeration (combining tasks to reduce overhead), and Mapping.

Distributed Memory Programming (MPI)

- Communication: Processes identified by ranks use Point-to-Point (MPI-Send, MPI-Recv) or Collective operations.

Collectives like MPI-Bcast, MPI-Reduce, and MPI-Allreduce have no tags and are matched by the order of calls in a communication.

- Data and Safety: Derived Datatypes (like MPI-Type-indexed) allow sending non-contiguous data. Asynchronous communication (MPI-Send, MPI-Irecv) enables overlapping computation with communication, checked via MPI-Wait or MPI-Test. Deadlocks can arise from improper buffering or circular dependencies, often solved by MPI-Sendrecv or MPI-Ssend.

OpenMP

OpenMP is a directives-based API for shared memory programming that uses a team of threads (Master and workers).

- Directives: Uses pragmas like `#pragma omp parallel` to fork threads. `parallel for` is used to parallelize loops, provided there are no loop-carried dependencies.

- Data Scoping: Variables are identified as shared or private. Reductions allow for safe global sums/products.

- Schedules: Load balancing is managed via static (fixed pre-assignment) or dynamic (first-come, first served) schedules.

- Tasks: Useful for irregular structures like traversing a linked list.

Interconnect Metrics

• Bisection Width:

- Linear Array = 1
- 2D Mesh = $\sqrt{p}$
- 2D Wraparound Mesh = $2\sqrt{p}$
- Hypercube = $p/2$
- Completely connected = $p^2/4$
- Star = 1

• Network Diameter

- Linear Array = $p-1$
- Ring = $\lceil p/2 \rceil$
- 2D Mesh = $2(\sqrt{p}-1)$
- 2D Wraparound Mesh = $2(\sqrt{p}/2)$
- Hypercube = $\log_2 p$
- Completely Connected = 1
- Star = 2

• Network Cost

- Linear Array/Star = $p-1$
- 2D Mesh (No wraparound) = $2(p-\sqrt{p})$
- Hypercube = $(p \log_2 p)/2$
- Completely Connected = $p(p-1)/2$

Communication Cost Models

- Per-word transfer time: $t_w = 1/R$, R (byte/sec)
- Packet Routing Time: $t_{comm} = t_s + t_{w1} m + t_{h1} l + t_{w2} (r+s) l + (m/r - 1) t_{w2} (r+s)$, r (messag bytes per packet) s (overhead bytes)
- Cut through routing time: $t_{comm} = t_s + t_{h1} + t_w m$
- Simplified Cost Model (when t_h is negligible): $t_{comm} = t_s + t_w m$

Performance Metrics

• Speedup: $S(n, p) = \frac{T_{serial}(n)}{T_{parallel}(n, p)}$

• Efficiency: $E(n, p) = \frac{S(n, p)}{p} = \frac{T_{serial}(n)}{p T_{parallel}(n, p)}$